Wrapping AWS Lambda ESM `handler`

The problem: How to transform AWS Lambda `handler` below?

```
// input.js
export const handler = async(event) => {
    return "Hi from AWS Lambda";
};
```

To the following, notice the `WrapAwsLambda` wrapper:

```
// transformed.js
export const handler = WrapAwsLambda(async(event) => {
    return "Hi from AWS Lambda";
});
```

Using loader hooks

Built-in Node.js mechanism to hook into the ESM `import`:

https://nodejs.org/api/module.html#customization-hooks

Asynchronous hooks

```
// register_async_hooks.mjs
import { register } from "node:module";
register("./async-hooks.mjs", import.meta.url);
```

Add `--import` to register hooks:

```
node --import ./register_async_hooks.mjs runtime.mjs
```

The hooks `load` function runs on a dedicated thread!

Async hooks `load` function

```
// async_hooks.mjs
import { transformLambda } from "./myTransform.js";
let patched = false;
export async function load(url, context, nextLoad) {
    const result = await nextLoad(url, context);
    if (!patched && url.endsWith("/handler.mjs")) {
        patched = true;
        return {
            format: "module", shortCircuit: true,
            source: transformLambda(result.source.toString(), "handler", "WrapAwsLambda")
        };
    }
    return result;
}
```

Note: async hooks are usually 2x slower than sync hooks.

Synchronous hooks `load` function

```
// sync_hooks.mjs
import { registerHooks } from "node:module";
import { transformLambda } from "./myTransform.js";
let patched = false;
registerHooks({
  load(url, context, nextLoad) {
    const result = nextLoad(url, context);
    if (!patched && url.endsWith("/handler.mjs")) {
      patched = true;
      return {
        format: "module",
        shortCircuit: true,
        source: transformLambda(result.source.toString(), "handler", "WrapAwsLambda")
      };
    }
    return result;
  },
});
```

Transforming ESM source code

Naive approach using `RegExp()` for direct string manipulation:

```ts
// regex-transform.ts
export function transformLambda(input: string, handler: string, wrapper: string): string {
  return input.replace(
    new RegExp(`export const ${handler} = (.+);`, "s"),
    `export const ${handler} = ${wrapper}($1);`
  );
}
```

This is the fastest way, which we will use as baseline for AST transformations:

`node --import ./sync-hooks-regex.mjs runtime.mjs => 26.6 ± 4.4 ms`

Babel transform

Babel transformation with custom `visitor` is used to generate the wrapper call:

```
// babel-transform.ts
ExportNamedDeclaration(path) {
  if (t.isVariableDeclaration(path.node.declaration)) {
    const varDecl = path.node.declaration.declarations[0];
    if (t.isIdentifier(varDecl.id) && varDecl.id.name === handler) {
      path.replaceWith(t.exportNamedDeclaration(
        t.variableDeclaration("const", [t.variableDeclarator(t.identifier(handler),
          t.callExpression(t.identifier(wrapper), [varDecl.init!]))]
      )));
      path.skip();
    }
  }
}
```

`node --import ./sync-hooks-babel.mjs runtime.mjs => 177.5 ± 6.6 ms`

Acorn => estraverse => astring

Acorn is only a parser, `estraverse` and `astring` add traversing and codegen.

```
const ast = acorn.parse(code, { ecmaVersion: 2020, sourceType: "module" });
estraverse.replace(ast as ESTree.Node, {
  enter: function (node) {
    if (node.type === "ExportNamedDeclaration" && node.declaration?.type === "VariableDeclaration") {
      const varDecl = node.declaration.declarations[0];
      if (varDecl.id.type === "Identifier" && varDecl.id.name === handler) {
        varDecl.init = NESTree.ESTree.CallExpression(NESTree.Helpers.AutoChain(wrapper),
          [varDecl.init as NESTree.ESTree.Expression]) as ESTree.Expression;
      }
      return { ...node };
    }
  }
});
return astring.generate(ast);
```

`node --import ./sync-hooks-acorn.mjs runtime.mjs => 48.7 ± 4.5 ms`

Rust native addon using `oxc.rs`

The oxc.rs tools and crate `oxc_transformer` provides API similar to Babel in Rust.

With napi.rs we can easily export the Rust code:

```rust
use napi_derive::napi;
mod transform;

#[napi]
pub fn transform_lambda(input: String) -> String {
  transform::transform_lambda_source(input)
}
```

```
const output = transformLambda(input);
```

`node --import ./sync-hooks-oxc.mjs runtime.mjs => 37.4 ± 2.2 ms`

Rust wasm plugin for `swc.rs`

Based on the SWC guide for plugin creation compiled to WebAssembly.

```
// swc-wrapper.cjs
const swc = require("@swc/core");
exports.transformLambda = function (sourceCode, handler, wrapper) {
  const output = swc.transformSync(sourceCode, {
    filename: "handler.mjs", sourceMaps: false, isModule: true, {
      target: "esnext", experimental: {
        plugins: [[ require.resolve("./my-plugin.wasm"), { handler, wrapper } ]]
      }
    },
  });
  return output.code;
};
```

`node --import ./sync-hooks-swc.mjs runtime.mjs => 42.5 ± 5.1 ms`

Conclusion

Asynchronous hooks are very slow, whenever possible use synchronous hooks.

Rust based solution beats pure JavaScript solution, but it is harder to write and maintain. From the JavaScript solutions the `acorn` + `astring` are very fast.

There is no big difference between `oxc.rs` and `swc.rs` in speed. Main difference is much bigger size of `swc.rs` package and caching to `.swc` folder which kills cold start.